Analisis exploratorio de los datos

Importamos librerias

[ ] ↳ *1 celda oculta*

Obtención de datos

[ ] ↳ *2 celdas ocultas*

Exploración de los DataFrame

[ ] ↳ *47 celdas ocultas*

Limpieza de los datos

1.Comenzamos con el df_ventas

Eliminamos los 000 de mas que se encuentran el la columan distributor, el punto final y la coma final despues de los valor 0

```
df_ventas = df_ventas.astype(str).apply(lambda x: x.str.rstrip('0').str.rstrip('.').str.rstrip(',').str.lstrip('$'))
df_ventas.head()
```

	distributor	Rubber	Brass	Vinyl	Granite	Stone	B
0	583	54.510.203,61	45.268.636,86	51.579.748,25	21.780.180,58	26.576.776,52	
1	1104	32.438.788,2	25.837.100,49	36.603.264,5	21.883.374,92	1.473.437,08	
2	1384	21.780.180,58	78.927.599,01	25.837.100,49	33.102.840,61	51.579.748,25	
3	379	79.358.855,35	90.185.311,22	45.268.636,86	54.510.203,61	59.358.855,35	
4	1599	11.758.005,07	21.780.180,58	57.187.306,41	9.945.371,16	32.067.534,68	

Próximos pasos: Generar código con df_ventas | Ver gráficos recomendados

Eliminamos el valor $

```
df_ventas = df_ventas.apply(lambda x : x.str.replace('$',''), axis = 1)
```

Reemplazamos la coma por el punto en los decimales, y eliminamos los puntos de los enteros

```
def reemplazar_puntos_comas(columna):
   if columna.name == 'distributor':  # Ignorar la primera columna
      return columna
   else:
      return columna.str.replace('.', '',).str.replace(',', '.')

```

```
df_ventas.iloc[:, 1:] = df_ventas.iloc[:, 1:].apply(reemplazar_puntos_comas)
```

```
df_ventas
```

	distributor	Rubber	Brass	Vinyl	Granite	
0	583	54510203.61	45268636.86	51579748.25	21780180.58	26576
1	1104	32438788.2	25837100.49	36603264.5	21883374.92	1473
2	1384	21780180.58	78927599.01	25837100.49	33102840.61	51579
3	379	79358855.35	90185311.22	45268636.86	54510203.61	59358
4	1599	11758005.07	21780180.58	57187306.41	9945371.16	32067
5	1062	90185311.22	43512922.94	11758005.07	76443998.86	36999
6	1679	78927599.01	57187306.41	41899590.44	18050385.12	20344
7	1302	5662736.92	59862809.01	46239695.3	90185311.22	3660
8	565	57009912.25	53172624.14	36999867.09	59199680.31	78927
9	860	36603264.5	54510203.61	1346769.34	18421797.13	10536
10	707	59199680.31	59199680.31	9945371.16	55872547.77	43512
11	846	61236075.66	32067534.68	21883374.92	53172624.14	59199
12	175	26576776.52	22300838.58	1473437.08	41572788.8	65978
13	241	59862809.01	55020982.8	78927599.01	55020982.8	54510
14	1553	33102840.61	21883374.92	43512922.94	177874267.01	3243
15	1093	1346769.34	16086016.33	65978582.24	12466621.84	11758
16	1666	45268636.86	79358855.35	59199680.31	26576776.52	33102
17	504	20344007.84	1346769.34	22300838.58	35563624.11	2704
18	723	77874267.01	51579748.25	32067534.68	11758005.07	57009
19	1723	42411396.23	57009912.25	90185311.22	57187306.41	77874
20	920	10536882.23	27179878.86	79358855.35	27009912.25	57187
21	1055	46239695.3	1473437.08	41572788.8	61236075.66	5662
22	815	51579748.25	46239695.3	53172624.14	27179878.86	25837
23	161	41572788.8	35563624.11	27179878.86	79358855.35	1346
24	378	16086016.33	11812951.32	18050385.12	51579748.25	21780
25	1235	35563624.11	42411396.23	81812951.32	65978582.24	90185
26	1017	22300838.58	61236075.66	61236075.66	36059867.09	61236
27	1463	21883374.92	18050385.12	59862809.01	25837100.49	9945
28	364	53172624.14	33102840.61	177874267.01	81812951.32	42411
29	1710	41899599.44	55872547.77	26576776.52	41899590.44	55072
30	1526	32067534.68	65978582.24	16086016.33	36603264.5	53172
31	234	11812951.32	11758005.07	18421797.13	43512922.94	16086
32	1560	57187306.41	177874267.01	54510203.61	32067534.68	3221
33	1489	1473437.08	76443998.86	42411396.23	22300838.58	4623
34	1183	55020982.8	9945371.16	20344007.84	41927599.01	41899
35	308	18050385.12	41899590.44	33102840.61	20344007.84	45268
36	1418	27179878.86	36999867.09	55872547.77	1346769.34	35563
37	818	25837100.49	26576776.52	21780180.58	6662942.92	51862
38	619	43512922.94	20344007.84	55020982.8	42411396.23	11812
39	715	9945371.16	41572788.8	6662942.92	1473437.08	5502
40	808	3221765.13	10536882.23	35563624.11	46039695.3	22300
41	325	36999867.09	6662942.92	76443998.86	45268636.86	18050
42	1169	55872547.77	18421797.13	12466621.84	59862809.01	21883
43	29	65978582.24	36603264.5	57009912.25	16931446.04	4157
44	nan	1680453941.34	1840659381	1912589120.61	1845294550.32	1651519

Próximos pasos: Generar código con df_ventas Ver gráficos recomendados

Eliminamos la última fila que contiene datos nulos

```
df_ventas.drop(df_ventas.index[44], inplace=True)
```

Cambiamos el tipo de dato de object a float64 de todas las columnas excepto la primera

```
for col in df_ventas.columns[1:43] : # para cada columna a partir de la columna 2(indice = 1)
    df_ventas[col] = df_ventas[col].astype('float')
```

```
df_ventas = df_ventas.astype(int)
```

```
df_ventas.head()
```

	distributor	Rubber	Brass	Vinyl	Granite	Stone	Brick	Aluminum	
0	583	54510203	45268636	51579748	21780180	26576776	0	55872547	18
1	1104	32438788	25837100	36603264	21883374	1473437	0	46239695	33
2	1384	21780180	78927599	25837100	33102840	51579748	0	21883374	78
3	379	79358855	90185311	45268636	54510203	59358855	0	79358855	32
4	1599	11758005	21780180	57187306	9945371	32067534	0	53172624	57

Próximos pasos: Generar código con df_ventas Ver gráficos recomendados

```
df_ventas.dtypes
```

```
distributor    int64
Rubber         int64
Brass          int64
Vinyl          int64
Granite        int64
Stone          int64
Brick          int64
Aluminum       int64
Glass          int64
Plexiglass     int64
Steel          int64
Wood           int64
Plastic        int64
dtype: object
```

2. Limpiamos el Df_distribuidores

Eliminamos los 000 de mas de la columna distributor y el punto final y la coma final después del valor 0

```
df_distribuidores = df_distribuidores.astype(str).apply(lambda x: x.str.rstrip('0').str.rstrip('.').str.rstrip(','))
df_distribuidores.head()
```

	id	distributor	distributor activities	years in the construction market
0	565	Abernathy-Hayes	construction materials import/distribution, ir...	15
1	1.384	Balistreri LLC	construction materials import/distribution, si...	22
2	1.183	Brekke-Stiedemann	construction materials import/distribution	13

Próximos pasos: Generar código con df_distribuidores Ver gráficos recomendados

Aplicamos la función para reemplazar las comas decimales por puntos

```
df_distribuidores.iloc[:, 0:] = df_distribuidores.iloc[:, 0:].apply(reemplazar_puntos_comas)
```

Elimine las columnas 9 y 32 que contenian datos nulos

```
df_distribuidores = df_distribuidores.drop(index=[9, 32])
df_distribuidores.head()
```

	id	distributor	distributor activities	years in the construction market
0	565	Abernathy-Hayes	construction materials import/distribution. ir...	15
1	1384	Balistreri LLC	construction materials import/distribution. si...	22
2	1183	Brekke-Stiedemann	construction materials import/distribution	13

Próximos pasos: Generar código con df_distribuidores Ver gráficos recomendados

Cambie de object a int64 las columnas de 'id' y 'years in the construction market'

```
df_distribuidores = df_distribuidores.astype({'id' : 'int64',
                                              'years in the construction market': 'int64'})
df_distribuidores.dtypes
```

```
id                                int64
distributor                      object
distributor activities           object
years in the construction market  int64
dtype: object
```

```
df_distribuidores.rename(columns={'distributor': 'namedist'}, inplace=True)
df_distribuidores.rename(columns={'id': 'distributor'}, inplace=True)
df_distribuidores.head()
```

	distributor	namedist	distributor activities	years in the construction market
0	565	Abernathy-Hayes	construction materials import/distribution. ir...	15
1	1384	Balistreri LLC	construction materials import/distribution. si...	22
2	1183	Brekke-Stiedemann	construction materials import/distribution	13

Próximos pasos: Generar código con df_distribuidores Ver gráficos recomendados

3. Limpiamos df_exportaciones

```
df_exportaciones.dtypes
```

```
distributor    float64
Rubber          object
Brass           object
Vinyl           object
Granite         object
Stone           object
Brick           object
Aluminum        object
Glass           object
Plexiglass      object
Steel           object
Wood            object
Plastic         object
Unnamed: 13    float64
Unnamed: 14     object
Columnas        object
dtype: object
```

Eliminamos los 000 de mas de la columna distributor y el punto final y la coma final después del valor 0

```
df_exportaciones = df_exportaciones.astype(str).apply(lambda x: x.str.rstrip('0').str.rstrip('.').str.rstrip(','))
df_exportaciones.head()
```

	distributor	Rubber	Brass	Vinyl	Granite	Stone	Bri
0	1.526	$22.431.099	$36.031.577	$31.118.167	$21.322.223	$35.382.848	$35.280.2
1	1.553	$27.566.922	$21.996.538	$39.412.316	$25.681.987	$41.861.783	$22.408.7
2	1.666	$37.577.095	$41.457.655	$31.467.967	$37.577.926	$35.845.106	$42.953.1
3	364	$36.012.730	$41.667.692	$22.837.073	$29.288.200	$39.553.494	$33.513.5
4	920	$43.416.417	$36.290.780	$23.679.738	$21.183.706	$25.210.622	$30.864.0

Próximos pasos: Generar código con df_exportaciones Ver gráficos recomendados

```
df_exportaciones.iloc[0:, 0:] = df_exportaciones.iloc[0:, 0:].apply(reemplazar_puntos_comas)
```

```
df_exportaciones['distributor'] = df_exportaciones['distributor'].apply(lambda x: x.replace('.', ''))
df_exportaciones.head()

```

	distributor	Rubber	Brass	Vinyl	Granite	Stone	Brick	Al
0	1526	$22431099	$36031577	$31118167	$21322223	$35382848	$35280292	$32
1	1553	$27566922	$21996538	$39412316	$25681987	$41861783	$22408742	$40
2	1666	$37577095	$41457655	$31467967	$37577926	$35845106	$42953168	$33
3	364	$36012730	$41667692	$22837073	$29288200	$39553494	$33513588	$36
4	920	$43416417	$36290780	$23679738	$21183706	$25210622	$30864041	$41

Próximos pasos: Generar código con df_exportaciones Ver gráficos recomendados

```
df_exportaciones = df_exportaciones.apply(lambda x : x.str.replace('$',''), axis = 1)
df_exportaciones.head()
```

	distributor	Rubber	Brass	Vinyl	Granite	Stone	Brick	Aluminum
0	1526	22431099	36031577	31118167	21322223	35382848	35280292	32362235
1	1553	27566922	21996538	39412316	25681987	41861783	22408742	40690302
2	1666	37577095	41457655	31467967	37577926	35845106	42953168	33817289
3	364	36012730	41667692	22837073	29288200	39553494	33513588	36827718
4	920	43416417	36290780	23679738	21183706	25210622	30864041	41173207

Próximos pasos: Generar código con df_exportaciones Ver gráficos recomendados

Eliminamos las columnas que contienen datos nulos

```
df_exportaciones.drop(df_exportaciones.columns[[13, 14, 15]], axis=1, inplace=True)
df_exportaciones.head()
```

	distributor	Rubber	Brass	Vinyl	Granite	Stone	Brick	Aluminum
0	1526	22431099	36031577	31118167	21322223	35382848	35280292	32362235
1	1553	27566922	21996538	39412316	25681987	41861783	22408742	40690302
2	1666	37577095	41457655	31467967	37577926	35845106	42953168	33817289
3	364	36012730	41667692	22837073	29288200	39553494	33513588	36827718
4	920	43416417	36290780	23679738	21183706	25210622	30864041	41173207

Próximos pasos: Generar código con df_exportaciones Ver gráficos recomendados

```
for col in df_exportaciones.columns[0:44] :
    df_exportaciones[col] = df_exportaciones[col].astype('int')
    df_exportaciones.dtypes
```

```
df_exportaciones.dtypes
```

```
distributor    int64
Rubber         int64
Brass          int64
Vinyl          int64
Granite        int64
Stone          int64
Brick          int64
Aluminum       int64
Glass          int64
Plexiglass     int64
Steel          int64
Wood           int64
Plastic        int64
dtype: object
```

```
df_exportaciones.drop_duplicates(inplace=True)
```

4. Limpiamos df_ciudadades

```
df_ciudades.head()
```

	PYid	id	location	department	activities
0	71	NaN	Abaí	Caazapá	food and beverage products manufacture, other ...
1	224	NaN	Acahay	Paraguarí	food and beverage products manufacture, other ...
2	208	NaN	Alberdi	Ñeembucú	food and beverage products manufacture, other ...
[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	food and beverage products manufacture, other

Próximos pasos: Generar código con df_ciudades Ver gráficos recomendados

```
df_ciudades.dropna(inplace=True) #eliminamos las filas que contienen datos nulos
```

```
df_ciudades = df_ciudades.reset_index(drop=True) #reseteamos el numero de índice
```

```
df_ciudades.drop(df_ciudades.columns[[0]], axis=1, inplace=True) #eliminamos la primera columna que contiene datos nulos
df_ciudades.head()
```

	id	location	department	activities
0	409.0	Altos	Cordillera	agriculture, livestock, hunting and related, l...
1	523.0	Coronel Martínez	Guairá	agriculture, livestock, hunting and related
2	519.0	Guayaybi	San Pedro	agriculture, livestock, hunting and related
3	286.0	Itapé	Guairá	agriculture, livestock, hunting and related, t...
4	112.0	San Cosme y Damián	Itapúa	agriculture, livestock, hunting and related, t...

Próximos pasos: Generar código con df_ciudades Ver gráficos recomendados

```
df_ciudades = df_ciudades.astype({'id' : 'int64'}) #cambiamos el tipo de dato de id a int64
df_ciudades.dtypes
```

```
id             int64
location      object
department    object
activities    object
dtype: object
```

```
df_ciudades.rename(columns={'id': 'distributor'}, inplace=True) #renombramos la columna id a distributor
df_ciudades
```

	distributor	location	department	activities
0	409	Altos	Cordillera	agriculture, livestock, hunting and related, l...
1	523	Coronel Martínez	Guairá	agriculture, livestock, hunting and related
2	519	Guayaybi	San Pedro	agriculture, livestock, hunting and related
3	286	Itapé	Guairá	agriculture, livestock, hunting and related, t...
4	112	San Cosme y Damián	Itapúa	agriculture, livestock, hunting and related, t...
5	169	San Miguel	Misiones	craft production
6	402	San Pedro del Paraná	Itapúa	agriculture, livestock, hunting and related

Próximos pasos: Generar código con df_ciudades Ver gráficos recomendados

Comenzamos las visualizaciones

```
import seaborn as sns
import matplotlib.pyplot as plt
```

```
# Creamos una nueva columna que sume los valores de todas las columnas excepto la primera
df_ventas['Total_ventas'] = df_ventas.iloc[:, 1:].sum(axis=1)
df_ventas.head()
```

	distributor	Rubber	Brass	Vinyl	Granite	Stone	Brick	Aluminum	
0	583	54510203	45268636	51579748	21780180	26576776	0	55872547	18
1	1104	32438788	25837100	36603264	21883374	1473437	0	46239695	33
2	1384	21780180	78927599	25837100	33102840	51579748	0	21883374	78
3	379	79358855	90185311	45268636	54510203	59358855	0	79358855	32
4	1599	11758005	21780180	57187306	9945371	32067534	0	53172624	57

Próximos pasos: Generar código con df_ventas Ver gráficos recomendados

```
df_exportaciones['Total_exportaciones'] = df_exportaciones.iloc[:, 1:].sum(axis=1)
df_exportaciones.head()
```

	distributor	Rubber	Brass	Vinyl	Granite	Stone	Brick	Aluminum
0	1526	22431099	36031577	31118167	21322223	35382848	35280292	32362235
1	1553	27566922	21996538	39412316	25681987	41861783	22408742	40690302
2	1666	37577095	41457655	31467967	37577926	35845106	42953168	33817289
3	364	36012730	41667692	22837073	29288200	39553494	33513588	36827718
4	920	43416417	36290780	23679738	21183706	25210622	30864041	41173207

Próximos pasos: Generar código con df_exportaciones Ver gráficos recomendados

Ahora vamos a generar una tabla unificando todos los datos relevantes para las visualizaciones ventas totales, exportaciones totales, numero de distribuidor y nombre.

```
df_dv = df_ventas[['distributor', 'Total_ventas']]
df_de = df_exportaciones[['distributor', 'Total_exportaciones']]
tabla_comparativa = df_dv.merge(df_de, on='distributor', how='left')
tabla_comparativa.head()
```

	distributor	Total_ventas	Total_exportaciones
0	583	507100047	365330063.0
1	1104	243275398	514646989.0
2	1384	350289311	386724998.0
3	379	526966383	399446640.0
4	1599	1443214732	397122970.0

Próximos pasos: Generar código con tabla_comparativa | Ver gráficos recomendados

```
tabla_comparativa.fillna(0, inplace=True)
tabla_comparativa.isnull().sum()
```

```
distributor            0
Total_ventas           0
Total_exportaciones    0
dtype: int64
```

```
dist_columns = df_distribuidores[['distributor', 'namedist']]
dist_columns.head()
```

	distributor	namedist
0	565	Abernathy-Hayes
1	1384	Balistreri LLC
2	1183	Brekke-Stiedemann
3	1526	Collins LLC
4	29	Cummings-Ward

Próximos pasos: Generar código con dist_columns | Ver gráficos recomendados

```
tabla_comparativa = tabla_comparativa.merge(dist_columns, on='distributor', how='right')
tabla_comparativa.head()

```

	distributor	Total_ventas	Total_exportaciones	namedist
0	565	4.330910e+08	4.023849e+08	Abernathy-Hayes
1	1384	3.502893e+08	3.867250e+08	Balistreri LLC
2	1183	4.554689e+08	1.263390e+10	Brekke-Stiedemann
3	1526	1.173773e+09	9.790751e+09	Collins LLC
4	29	8.337953e+08	4.090691e+08	Cummings-Ward

Próximos pasos: Generar código con tabla_comparativa | Ver gráficos recomendados

```
tabla_comparativa.fillna(0, inplace=True)
tabla_comparativa.isnull().sum()
```

```
distributor            0
Total_ventas           0
Total_exportaciones    0
namedist               0
dtype: int64
```

La información que tenemos de las ciudades no coincide con los distribuidores que realizan exportaciones y ventas

```
ciu_columns = df_ciudades[['distributor', 'location','department']]
ciu_columns.head()
```

	distributor	location	department
0	409	Altos	Cordillera
1	523	Coronel Martínez	Guairá
2	519	Guayaybi	San Pedro
3	286	Itapé	Guairá
4	112	San Cosme y Damián	Itapúa

Próximos pasos: Generar código con ciu_columns Ver gráficos recomendados

```
tabla_conciu = df_exportaciones.merge(ciu_columns, on='distributor', how='right')
tabla_conciu.head() #Los datos que tenemos de las ciudades no coinciden con los distribuidores que realizan exportaciones y ventas
```

	distributor	Rubber	Brass	Vinyl	Granite	Stone	Brick	Aluminum	Glass	Plexigla
0	409	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	N
1	523	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	N
2	519	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	N
3	286	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	N
4	112	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	N

Próximos pasos: Generar código con tabla_conciu Ver gráficos recomendados

```
tabla_comparativa['Total_ventas'] = tabla_comparativa['Total_ventas'].astype(int)
tabla_comparativa['Total_exportaciones'] = tabla_comparativa['Total_exportaciones'].astype(int)
tabla_comparativa.dtypes
```

```
distributor             int64
Total_ventas            int64
Total_exportaciones     int64
namedist               object
dtype: object
```

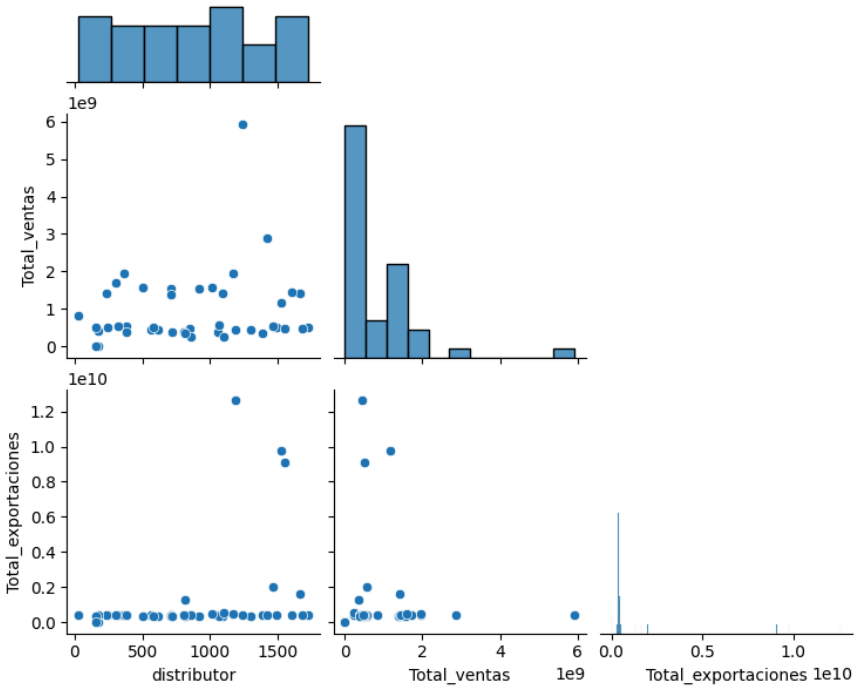
```
tabla_comparativa ['Total_ventas'] - tabla_comparativa ['Total_exportaciones'].astype(int)
```

```
0        30706104
1       -36435687
2    -12178431096
3     -8616978706
4       424726238
5       -21838981
6      1049921147
7      1183948165
8       192656133
9       127519743
10      109642304
11      -21644847
12     1121780499
13     1196859570
14     1054874641
15       71316865
16       47800925
17     -155693794
18       76687480
19      996158102
20    -1466794662
21      141769984
22     -271371591
23       27447697
24              0
25      161513466
26      160052357
27     -219234027
28    -8583110374
29       85562538
30     1569850876
31     1046091762
32     1302401341
33      -26099926
34     1114234764
35     -941293991
```

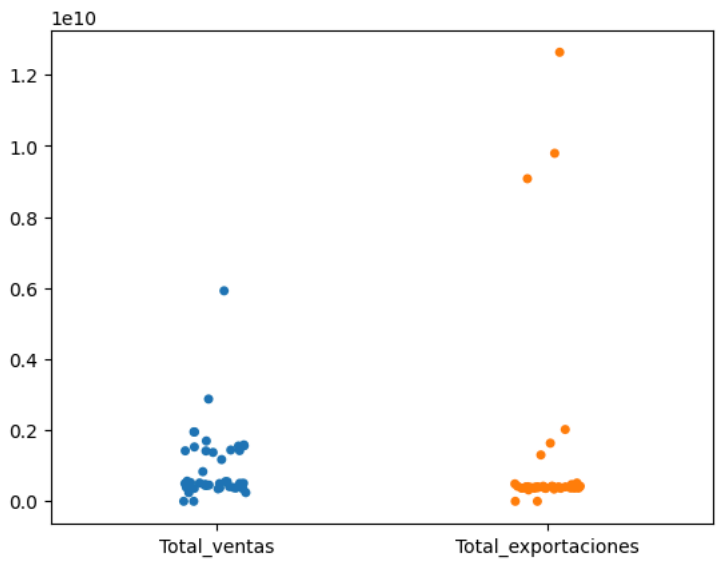
```
36        9971147
37       99159531
38        9330754
39     2454609946
40     5512260515
41              0
42     1458372104
43       79399265
dtype: int64
```

```
sns.pairplot(data = tabla_comparativa, corner = True)
```

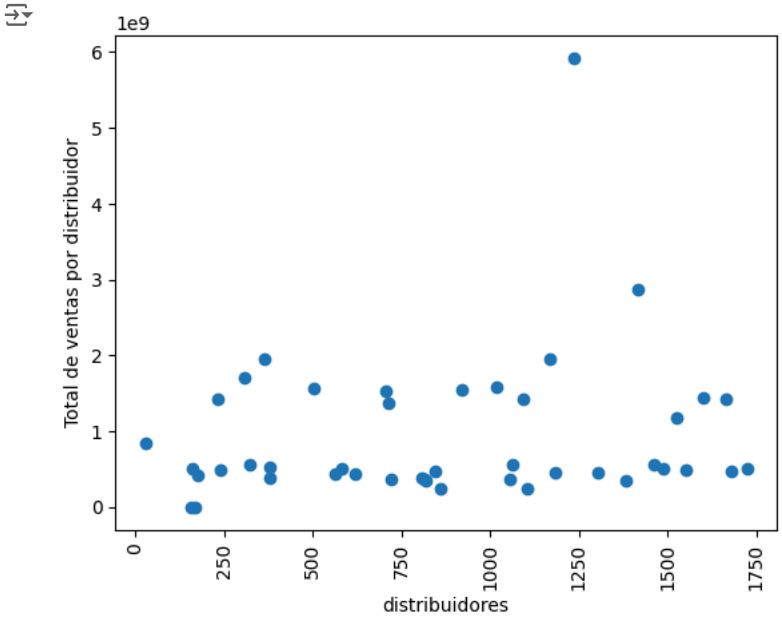
```
<seaborn.axisgrid.PairGrid at 0x7a8f792b9c30>
```



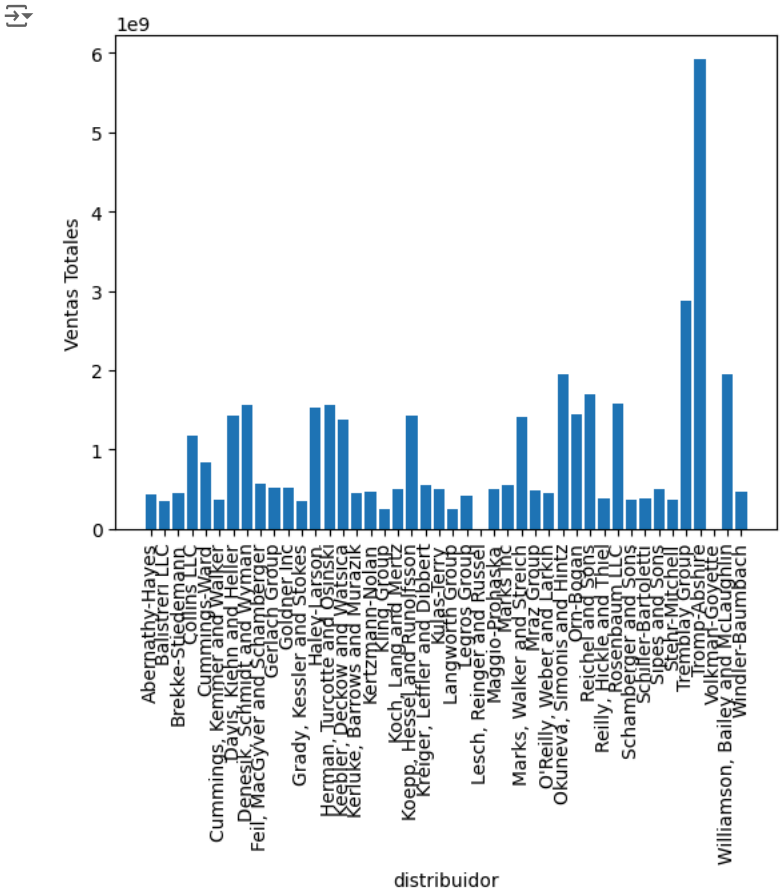

```
sns.stripplot(data = tabla_comparativa[['Total_ventas','Total_exportaciones']])
plt.show()
```



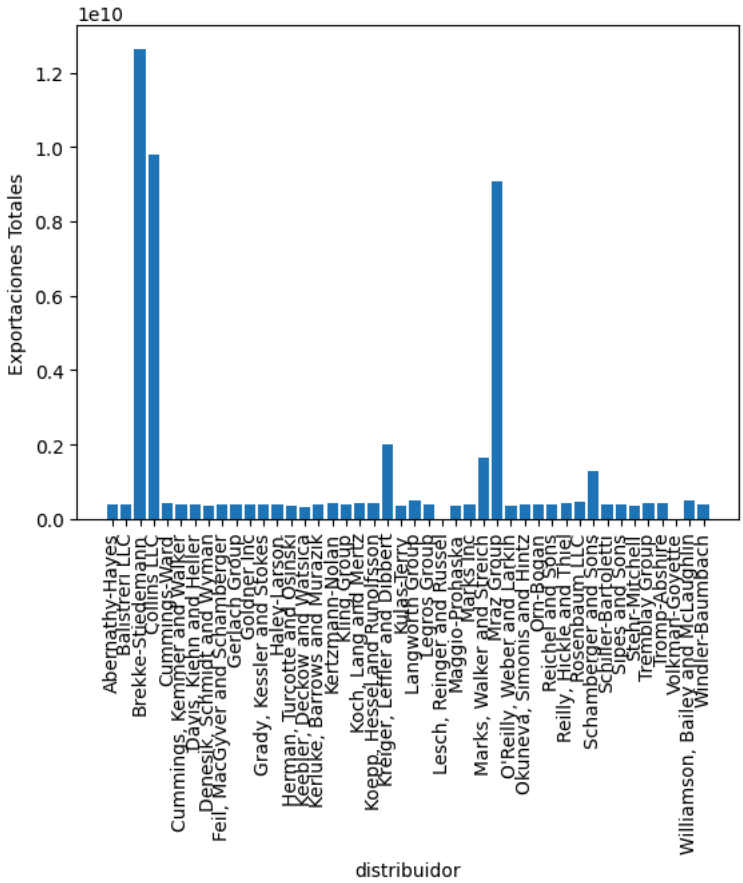
```
plt.scatter(tabla_comparativa['distributor'], tabla_comparativa['Total_ventas'])
plt.xlabel('distribuidores')
plt.ylabel('Total de ventas por distribuidor')
plt.xticks(rotation=90)
plt.show()
```




```
plt.bar(tabla_comparativa['namedist'], tabla_comparativa['Total_ventas'])
plt.xlabel('distribuidor')
plt.ylabel('Ventas Totales')
plt.xticks(rotation=90)
plt.show()
```




```
plt.bar(tabla_comparativa['namedist'], tabla_comparativa['Total_exportaciones'])
plt.xlabel('distribuidor')
plt.ylabel('Exportaciones Totales')
plt.xticks(rotation=90)
plt.show()
```



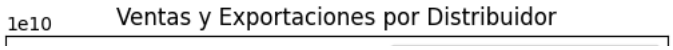

```
fig, ax = plt.subplots()
width = 0.70  # Ancho de las barras

# Posiciones de las barras
ind = range(len(tabla_comparativa))

# Barras de ventas
ax.bar(ind, tabla_comparativa['Total_ventas'], width, label='Ventas Totales')

# Barras de exportaciones
ax.bar([i + width for i in ind], tabla_comparativa['Total_exportaciones'], width, label='Exportaciones Totales')

# Etiquetas y título
ax.set_xlabel('Distribuidor')
ax.set_ylabel('Total')
ax.set_title('Ventas y Exportaciones por Distribuidor')
ax.set_xticks([i + width / 2 for i in ind])
ax.set_xticklabels(tabla_comparativa['namedist'])
plt.xticks(rotation=90)
ax.legend()
plt.show()
```


1e10
Ventas y Exportaciones por Distribuidor